

What Makes Cross-Variable Mixing Effective? Variable-Specific Value Interfaces for Multivariate Time-Series Forecasting

Anonymous submission

Abstract

Recent multivariate time-series forecasting methods increasingly exploit cross-variable dependencies through mixing mechanisms that determine which variables communicate and how strongly. However, variables often differ in periodicity, phase, lag, and predictive role. Selecting a useful source therefore does not determine how its information should be expressed for different targets, yet existing mixers typically aggregate values in a shared latent space. We revisit cross-variable aggregation through variable-specific value interfaces. Our FRAMEMIX operator equips variables with learnable orthogonal value interfaces for expressing outgoing information and interpreting aggregated information. Composing the source and receiver interfaces yields a distinct value transformation for every ordered variable pair without directly learning pairwise operators. We incorporate FrameMix into FRAMEFORMER, a patch-based architecture that mixes variables at each patch position before modeling temporal dependencies within each variable. Experiments show that variable-specific interfaces improve multiple routing mechanisms and remain effective with both learned and fixed frames, indicating that their gains cannot be explained by additional trainable capacity alone. More expressive pair-specific alternatives require substantially more parameters but provide no reliable accuracy gains. Across eight benchmark datasets and forecast horizons from 96 to 720, Frameformer outperforms established baselines in most of the evaluated settings.

1 Introduction

Multivariate time-series (MVTs) forecasting increasingly relies on cross-variable mechanisms—including variate attention, learned graphs, and explicit cross-dimension stages—to exchange information across variables (Liu et al. 2024; Wu et al. 2020; Cao et al. 2020; Zhang and Yan 2023). Yet the benefit of such mixing remains inconsistent. Channel-independent models often match or outperform architectures designed for cross-variable interaction (Zeng et al. 2023; Nie et al. 2023), suggesting that explicitly modeling inter-variable dependencies is not always beneficial. However, this pattern is not universal. In many forecasting tasks with strongly coupled variables, removing cross-variable communication sub-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

		Weather (no-mix: 0.168)				PEMSD8 (no-mix: 0.376)			
Routing	Rand.	.172	.151	.159	.153	.251	.209	.195	.181
	Corr.	.164	.149	.151	.149	.250	.208	.192	.181
	Simi.	.174	.153	.153	.156	.260	.209	.206	.181
	Atten.	.172	.150	.157	.154	.257	.202	.198	.176
		Identity	Haar	Hadamard	Learned	Identity	Haar	Hadamard	Learned
		Frame				Frame			

Figure 1: Variable-specific frames consistently improve forecasting performance across diverse routing mechanisms (lower/darker is better). Replacing shared-space mixing (Identity) with any non-identity frame reduces error for nearly every routing rule on both datasets. Values in parentheses indicate the channel-independent (no cross-variable mixing) baseline.

stantially degrades performance. These seemingly conflicting findings suggest that the central question is not simply whether cross-variable mixing helps, but which component of the mixing mechanism actually carries its predictive benefit when mixing is useful.

Most existing methods answer this question through *routing*: determining which variables communicate and how strongly each source contributes. Attention weights, graph edges, and related routing mechanisms select the information received by each variable, while the communicated values are typically processed in a representation space shared across all variables. This implicitly assumes that, once a useful source has been selected, its representation can be delivered to different receivers in essentially the same form.

However, variables in multivariate time series often differ in physical meaning, periodicity, phase, scale, lag, and predictive role. As a result, information from the same source variable can play different roles for different receivers. For example, a traffic sensor may provide an early warning for one road segment, reflect congestion at another with a delay, and be largely irrelevant to a third. Consequently, the same source information may need to be expressed differently for different receivers. Routing determines *who communicates* and *with what strength*, but it does not explicitly determine *how communicated values should be expressed by the source*

and interpreted by the receiver. We refer to this second design axis as a *variable-specific value interface*.

Motivated by this perspective, we introduce FRAMEMix, a cross-variable mixing operator that realizes variable-specific value interfaces through learnable orthogonal frames. Each variable owns a frame that expresses its outgoing values and interprets incoming aggregated information. Composing the source and receiver frames produces an effective transformation for every ordered source–receiver pair, enabling pair-specific value processing without requiring an independent transformation for every variable pair while leaving the routing mechanism unchanged. Figure 1 illustrates the central observation motivating our approach: replacing conventional shared-space mixing with variable-specific frames consistently improves forecasting performance across diverse routing mechanisms.

To isolate the role of value interfaces, we compare matched models that share the same backbone and routing mechanism but differ only in their frame construction. Setting all frames to the identity exactly recovers conventional shared-space mixing. Across multiple routing mechanisms, introducing non-identity frames consistently improves forecasting accuracy, including settings where shared-space cross-variable mixing provides little or no benefit. Conversely, continually permuting routing assignments causes only minor performance degradation, whereas disrupting the correspondence between variables and frames removes most of the frame gain. Further analyses show that fixed Haar-random and signed-permuted Hadamard frames achieve substantial improvements without trainable frame parameters, while substantially more expressive pair-specific parameterizations provide little additional benefit. Together, these results indicate that persistent variable-specific value interfaces, rather than exact routing, account for much of the predictive benefit of cross-variable aggregation. Under a common training protocol, Frameformer equipped with FrameMix achieves the lowest mean forecasting error on six of eight benchmark datasets while using substantially fewer parameters than existing cross-variable models.

Our contributions are threefold:

- **Identifying a second design axis.** We distinguish cross-variable aggregation into routing and variable-specific value interfaces, and show that the latter provides predictive gains beyond routing alone.
- **FrameMix.** We introduce FrameMix, a cross-variable mixing operator that realizes variable-specific value interfaces through learnable orthogonal frames, enabling pair-specific value processing without pair-specific transformations. We integrate FrameMix into the patch-based FRAMEFORMER architecture.
- **Controlled evidence.** Through controlled analyses over frame presence, construction, trainability, ownership persistence, and parameterization, we show that the observed gains cannot be explained by additional trainable capacity alone and instead arise from persistent variable-specific value interfaces.

2 Related Work

Multivariate time-series forecasting. Recent multivariate forecasting methods differ fundamentally in how they treat the variable dimension. Channel-independent methods such as DLinear (Zeng et al. 2023) and PatchTST (Nie et al. 2023) process each variable separately, providing strong performance while avoiding spurious cross-variable dependencies. Cross-variable methods instead allow information exchange through attention, learned graphs, or explicit mixing stages. iTransformer (Liu et al. 2024) applies attention across variable tokens, Crossformer (Zhang and Yan 2023) introduces a dedicated cross-dimension stage, and graph-based forecasters (Wu et al. 2020; Cao et al. 2020; Yi et al. 2023) learn inter-variable adjacencies for message passing. Recent hybrid methods seek to retain the robustness of channel independence while introducing selective cross-variable interaction.

Among cross-variable methods, interaction has been formulated primarily through routing: determining which variables should communicate and how strongly. Accordingly, prior comparisons have largely focused on whether cross-variable mixing is beneficial or which routing mechanism performs best. We instead decompose cross-variable aggregation into two components: routing, which selects and combines sources, and variable-specific value interfaces, which determine how information is expressed by each source and interpreted by each receiver. Several recent methods implicitly modify how communicated values are processed, suggesting that this second component may influence forecasting performance. CARD (Wang et al. 2024a) introduces channel-aligned attention, LIFT (Zhao and Shen 2024) explicitly adjusts lead–lag relationships, and CI/mixing hybrids such as CSformer (Wang et al. 2025) add lightweight cross-variable paths to channel-independent backbones. Because these methods alter multiple aspects of the forecasting architecture together, however, their predictive gains do not isolate routing, additional capacity, variable-specific processing, or persistent variable–parameter correspondence. Our matched interventions isolate these factors through controlled comparisons, allowing the contribution of variable-specific value interfaces to be isolated from routing and model capacity.

Sheaf-based transport. Sheaf neural networks associate local representation spaces with graph entities and use linear maps to transport information between them. Sheaf attention (Barbero et al. 2022b) and connection-Laplacian methods (Barbero et al. 2022a) incorporate such transport into graph propagation, while Bundle Neural Networks (Bamberger et al.) assign orthogonal maps to nodes and use the resulting flat-bundle structure for graph diffusion. PolyNSD (Borgi, Silvestri, and Liò 2025) uses polynomial spectral filters over the sheaf Laplacian to obtain multi-hop propagation even with diagonal restriction maps. Concurrent work on temporal link prediction (Asif, Khan, and Khan 2026) uses event-updated Householder frames to transport node memories between local coordinate systems.

FrameMix brings this transport principle to a different question. It places variable-specific frames only in the value path of cross-variable aggregation, leaves the routing mechanism unchanged, and thereby isolates the contribution of variable-specific value interfaces to multivariate forecasting.

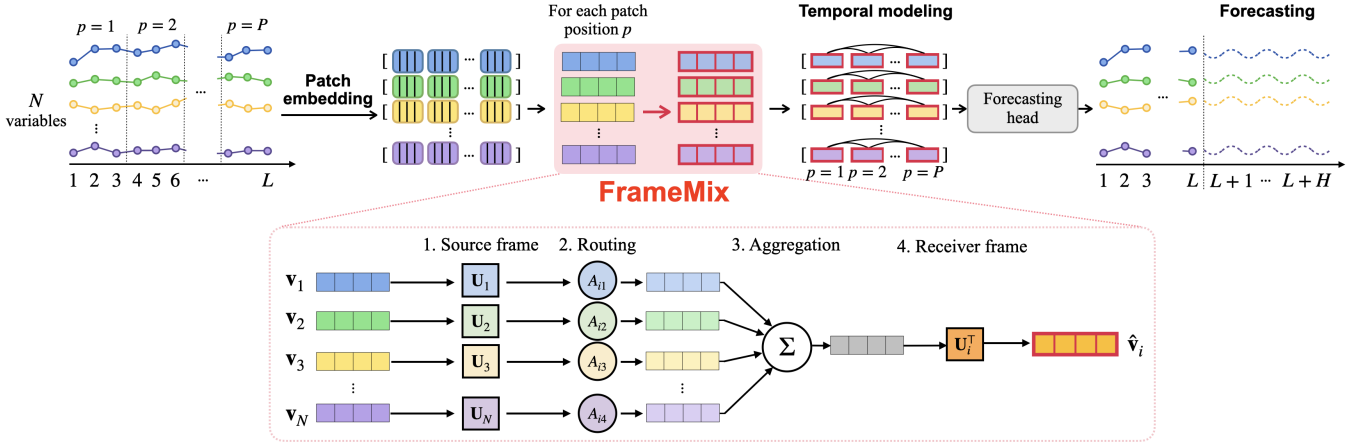


Figure 2: An overview of the Frameformer.

3 Our Approach

We present *Frameformer*, a patch-based forecasting architecture that improves cross-variable aggregation through variable-specific value interfaces. As illustrated in Figure 2, Frameformer consists of patch embedding, cross-variable mixing, temporal modeling, and forecasting stages. Each variable is divided into temporal patches and embedded as a sequence of tokens. At each patch position, variables exchange information through the proposed FrameMix operator, after which a channel-independent temporal Transformer models dependencies across patches within each variable. A forecasting head then produces the future sequence.

We first formulate the forecasting problem in Section 3.1. In Section 3.2, we introduce source and receiver interfaces that add a second form of pair specificity beyond the scalar routing weights. Section 3.3 introduces the FrameMix operator, and Section 3.4 describes its integration into the complete Frameformer architecture.

3.1 Problem Setup

Let $\mathbf{X} \in \mathbb{R}^{L \times N}$ denote a multivariate time series containing N variables observed over the past L time steps. The forecasting task is to predict the next H time steps, $\mathbf{Y} \in \mathbb{R}^{H \times N}$.

A patch-based forecaster partitions the history of each variable into P temporal patches and embeds them into latent tokens. Let $\mathbf{v}_i^{(p)} \in \mathbb{R}^d$ denote the value representation of variable i at patch position p . At each patch position, a cross-variable mixer produces an updated representation $\hat{\mathbf{v}}_i^{(p)}$ by aggregating information from the other variables. Temporal modeling is then performed across patch positions within each variable before the forecasting head generates the final predictions.

Our focus is the cross-variable aggregation operator. We modify how value representations are exchanged across variables while leaving the temporal modeling and forecasting stages otherwise unchanged.

3.2 Routing and Value Interfaces

In multivariate time series, variables exhibit heterogeneity: they differ in periodicity, phase, and lag structure. These differences may require their latent representations to be expressed differently when information is exchanged across variables, whereas conventional cross-variable aggregation typically combines them in a shared representation space. Scalar routing weights can control the magnitude of each source contribution but cannot change how that contribution is represented. We accordingly write cross-variable aggregation as

$$\hat{\mathbf{v}}_i = \mathbf{T}_i \sum_{j=1}^N A_{ij} \mathbf{S}_j \mathbf{v}_j, \quad (1)$$

where A_{ij} is the scalar routing weight from source variable j to receiver variable i , $\mathbf{S}_j \in \mathbb{R}^{d \times d}$ is a source-specific value map, and $\mathbf{T}_i \in \mathbb{R}^{d \times d}$ is a receiver-specific value map.

We use *routing* to refer broadly to the mechanism that determines the weights A_{ij} , including fixed rules, similarity or correlation scores, static learned mixing matrices, and learned attention rules. Routing therefore specifies which sources contribute and how strongly. In contrast, the value maps specify the coordinate convention in which a source expresses its information and the form in which a receiver interprets the resulting mixture.

Existing cross-variable mixers differ mainly in how the pairwise weights are obtained: learned attention over variate tokens, a static learned channel-mixing matrix, or a correlation-based graph. In many commonly used mixers, however, values are aggregated through a shared value representation between variables and do not contribute pair specificity, which corresponds to the setting $\mathbf{S}_j = \mathbf{T}_i = \mathbf{I}$ and yields

$$\hat{\mathbf{v}}_i = \sum_{j=1}^N A_{ij} \mathbf{v}_j. \quad (2)$$

In this case, ordered-pair specificity enters only through the scalar weight A_{ij} , which applies the same rescaling to every

coordinate and cannot provide receiver-specific coordinate-wise processing.

By introducing source and receiver interfaces, Eq. (1) adds a second form of pair specificity: the composed map $\mathbf{T}_i \mathbf{S}_j$ determines how source j 's contribution is re-expressed for receiver i . Because this pairwise map is composed from one interface per variable, it enables source–receiver-specific processing without learning an independent transformation for every ordered pair. We next instantiate these interfaces using variable-specific orthogonal frames.

3.3 The FrameMix Operator

We propose FrameMix, which realizes variable-specific value interfaces by assigning each variable a learnable orthogonal frame. Specifically, Eq. (1) is instantiated by setting $\mathbf{S}_j = \mathbf{U}_j$ and $\mathbf{T}_i = \mathbf{U}_i^\top$, yielding

$$\begin{aligned} \hat{\mathbf{v}}_i &:= \text{FrameMix}_i(\{\mathbf{v}_j\}_{j=1}^N; \mathbf{A}, \mathcal{U}) \\ &= \mathbf{U}_i^\top \sum_{j=1}^N A_{ij} \mathbf{U}_j \mathbf{v}_j = \sum_{j=1}^N A_{ij} \underbrace{(\mathbf{U}_i^\top \mathbf{U}_j)}_{\mathbf{R}_{ij}} \mathbf{v}_j. \end{aligned} \quad (3)$$

where $\mathcal{U} = \{\mathbf{U}_i\}_{i=1}^N$ and $\mathbf{R}_{ij} = \mathbf{U}_i^\top \mathbf{U}_j$ is the effective value transformation associated with the ordered source–receiver pair (i, j) .

The routing weight A_{ij} determines how strongly source variable j contributes, whereas $\mathbf{R}_{ij}$ determines how that contribution is represented for receiver variable i . Since each pairwise transformation is constructed from the corresponding source and receiver frames, FrameMix realizes an effective transformation for every ordered source–receiver pair while learning only one frame per variable, rather than one transformation for every variable pair. The attention query and key computations remain unchanged; FrameMix modifies only the value aggregation path.

Each frame is parameterized as a product of k Householder reflections,

$$\mathbf{U}_i = \prod_{\ell=1}^k \mathcal{H}(\mathbf{h}_{i,\ell}), \quad \mathcal{H}(\mathbf{h}) = \mathbf{I} - \frac{2\mathbf{h}\mathbf{h}^\top}{\mathbf{h}^\top \mathbf{h}}. \quad (4)$$

Here, $\mathbf{h}_{i,\ell} \in \mathbb{R}^d$ is the learnable vector parameterizing the ℓ -th Householder reflection for variable i . This parameterization guarantees orthogonality, $\mathbf{U}_i^\top \mathbf{U}_i = \mathbf{I}$, while requiring $O(kd)$ parameters per variable.

3.4 Frameformer architecture

Our Frameformer places FrameMix between patch embedding and temporal modeling. Given a multivariate time series, each variable is first converted into a sequence of patch tokens. FrameMix then performs cross-variable aggregation independently at each patch position. The resulting tokens are processed across patch positions within each variable, and a forecasting head maps the temporal representations to the prediction horizon.

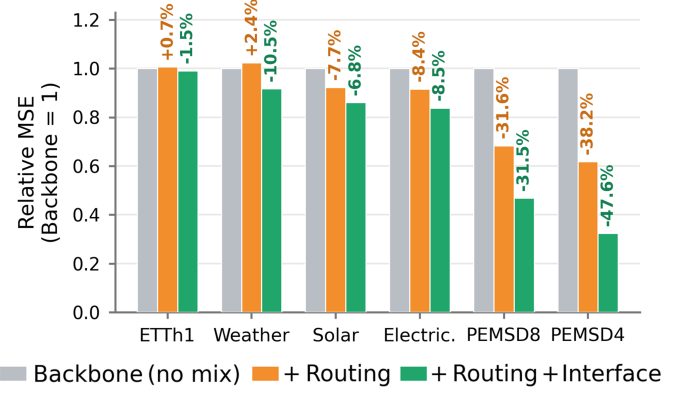


Figure 3: Stagewise gains from routing and variable-specific interfaces. MSE is normalized by the no-mixing backbone; lower is better.

Patch embedding. For each variable i , the input history $\mathbf{x}_i \in \mathbb{R}^L$ is divided into P non-overlapping patches of length s , where $P = L/s$. A shared linear projection maps the p -th patch to a d -dimensional token:

$$\mathbf{v}_i^{(p)} = \mathbf{W}_{\text{patch}} \mathbf{x}_i^{(p)} + \mathbf{e}^{(p)}, \quad (5)$$

where $\mathbf{x}_i^{(p)} \in \mathbb{R}^s$ is the p -th patch of variable i and $\mathbf{e}^{(p)}$ is its positional embedding.

Cross-variable FrameMix. At each patch position p , FrameMix is applied across the N variable tokens:

$$\hat{\mathbf{v}}_i^{(p)} = \text{FrameMix}_i(\{\mathbf{v}_j^{(p)}\}_{j=1}^N; \mathbf{A}^{(p)}, \mathcal{U}^{(p)}), \quad (6)$$

where $\mathbf{A}^{(p)} = [A_{ij}^{(p)}]_{i,j=1}^N$ is the routing-weight matrix and $\mathcal{U}^{(p)} = \{\mathbf{U}_i^{(p)}\}_{i=1}^N$ is the set of variable-specific frames. The routing weights are computed by the original query–key branch, while the frames modify only the value path.

Temporal modeling. For each variable i , the FrameMix outputs across patch positions are stacked into $\hat{\mathbf{V}}_i = [\hat{\mathbf{v}}_i^{(1)}, \dots, \hat{\mathbf{v}}_i^{(P)}]^\top$. A channel-independent temporal Transformer (Vaswani et al. 2017) processes this sequence across patches, producing $\mathbf{Z}_i$.

Forecasting head. A shared forecasting head maps $\mathbf{Z}_i$ to $\hat{\mathbf{y}}_i \in \mathbb{R}^H$, and the variable-wise predictions are collected as $\hat{\mathbf{Y}} = [\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_N] \in \mathbb{R}^{H \times N}$.

Training objective. Frameformer is trained end-to-end by minimizing the mean squared forecasting error:

$$\mathcal{L} = \frac{1}{HN} \sum_{t=1}^H \sum_{i=1}^N \left(Y_{t,i} - \hat{Y}_{t,i} \right)^2. \quad (7)$$

4 Analyzing Cross-Variable Value Interfaces

Section 3.2 separates cross-variable aggregation into routing and value interfaces. In this section, we first measure the

	Identity	Haar	Hadamard	FrameMix
Weather	.172	.150	.157	.154
Solar	.192	.205	.179	.179
Electricity	.153	.144	.144	.140
PEMSD8	.257	.202	.198	.176
PEMSD4	.212	.152	.144	.111

Table 1: Test MSE of the Frameformer and its variants using different frames.

stagewise contribution of each component, then test whether the gains from frames persist across different routing and frame choices, and finally examine whether orthogonality itself is necessary for these gains.

4.1 Routing and Frames Make Stagewise Contributions

We evaluate a nested within-host ablation with three cumulative conditions. (i) *Backbone* disables cross-variable aggregation ($\mathbf{S}_j = \mathbf{T}_i = \mathbf{I}$, $A_{ij} = \delta_{ij}$). (ii) *Backbone + routing* adds attention-based cross-variable aggregation while retaining the shared value space ($\mathbf{S}_j = \mathbf{T}_i = \mathbf{I}$). (iii) *Backbone + routing + frame* further adds variable-specific frames to the same routed mixer.

Figure 3 separates the contribution of routing from the additional contribution of frames. On Solar and Electricity, routing first reduces MSE by 7.7%–8.4%, and adding frames provides a further 6.8%–8.5% reduction. Both stages contribute more strongly on PEMS8 and PEMS4: routing reduces MSE by 31.6% and 38.2%, respectively, followed by additional frame reductions of 31.5% and 47.6%.

Weather reveals a different regime. Adding routing alone slightly increases error, yet adding frames to the same routed backbone makes the final model beneficial relative to the backbone. ETTh1 serves as a negative control, with little net change across the three conditions. Thus, frames neither replace nor duplicate routing. They can provide additional gains when routing already helps, or make an otherwise ineffective routed backbone beneficial.

4.2 Frame Gains Persist Across Routing and Frame Choices

The stagewise comparison in Section 4.1 uses a learned attention router and learned frames. We next test whether the frame contribution depends on either choice.

Figure 1 compares shared-space aggregation with fixed Haar, fixed Hadamard, and learned frames under four routing rules. Across the routing rules considered, every non-identity frame improves over its matched shared-space counterpart. On Weather, frames make mostly ineffective routers beneficial. On PEMS8, they add further gains to routers that already help.

Table 1 extends the construction comparison across additional datasets. Hadamard improves over Identity on every displayed dataset, while Haar does so on all but Solar. Learned frames provide their clearest additional gains on

	Identity	Affine (non-orth.)	FrameMix (orth.)
Weather	.172	.157	.154
Solar	.192	.187	.179
Electricity	.153	.139	.140
PEMSD8	.257	.181	.176
PEMSD4	.212	.120	.111

Table 2: Test MSE of Frameformer and its non-orthogonal affine-interface variant.

PEMS8 and PEMS4. Thus, the frame benefit is not tied to one learned router or to additional trainable frame parameters.

4.3 Are Orthogonal Frames Necessary?

FrameMix uses tied orthogonal source and receiver interfaces. We next ask whether the gain requires this specific constraint, or whether variable-specific value processing is the broader source of improvement. We replace the orthogonal maps with per-variable affine functions, $\mathbf{S}_j(\mathbf{v}) = \mathbf{s}_j^{\text{src}} \odot \mathbf{v} + \mathbf{b}_j^{\text{src}}$ and $\mathbf{T}_i(\mathbf{z}) = \mathbf{s}_i^{\text{tgt}} \odot \mathbf{z} + \mathbf{b}_i^{\text{tgt}}$, while preserving the same routing weights and value-path placement.

As shown in Table 2, the affine variant recovers 91%–94% of the Identity-to-FrameMix improvement on the PEMS datasets and 83% on Weather. It slightly outperforms orthogonal FrameMix on Electricity, whereas on Solar it recovers only 38% of the gain. These results indicate that variable-specific interfaces account for much of the improvement in several datasets, while orthogonal structure provides additional benefits in selected regimes. FrameMix remains a useful instantiation: it preserves value norms, ties the source and receiver maps through a single frame per variable, and achieves the best performance among these variants on Weather, Solar, and both PEMS datasets.

5 Experiments

The preceding analyses show that variable-specific interfaces improve cross-variable mixing across routing mechanisms and frame constructions. We now evaluate whether these gains translate into end-to-end forecasting performance and examine the regimes in which they occur.

Datasets and evaluation. We evaluate Frameformer on eight multivariate forecasting benchmarks: ETTh1, ETTh2, Exchange, Weather, Solar, PEMS8, PEMS4, and Electricity. The primary evaluation uses a forecast horizon of $h = 96$, with additional results at $h \in \{192, 336, 720\}$ reported in Section 5. All methods use the same data loaders, splits, and training-set normalization. We report test MSE as the three-seed mean and standard deviation.

Baselines. We compare Frameformer with DLinear (Zeng et al. 2023), PatchTST (Nie et al. 2023), iTransformer (Liu et al. 2024), TimeXer (Wang et al. 2024b), Crossformer (Zhang and Yan 2023), CARD (Wang et al. 2024a), and CSformer (Wang et al. 2025). Model-specific architectural settings follow the corresponding papers. Because CS-

Model	ETTh1	ETTh2	Exch.	Weather	Solar	PEMSD8	PEMSD4	Electric.
DLinear	.463±.003	.244±.004	.106±.003	.198±.002	.283±.001	.652±.008	.425±.001	.196±.000
PatchTST	.467±.010	.250±.004	.115±.006	.176±.003	.218±.017	.435±.003	.507±.015	.170±.003
iTransformer	.483±.016	.248±.001	.114±.001	.176±.002	.206±.002	.376±.008	.322±.004	.154±.000
TimeXer	.465±.006	.240±.002	.113±.002	.164±.002	.199±.008	.269±.018	.158±.001	.142±.002
Crossformer	.467±.003	.473±.081	1.075±.166	.165±.002	.194±.009	.224±.005	.112±.003	.158±.009
CARD	.496±.017	.276±.010	.155±.015	.163±.003	.208±.005	.265±.031	.182±.012	.146±.001
CSformer	.470±.007	.234±.003	.110±.004	.167±.005	.203±.007	.358±.003	.285±.028	.148±.003
Frameformer	.455±.004	.235±.002	.115±.007	.154±.002	.179±.005	.176±.002	.111±.001	.140±.003

Table 3: Test MSE on eight multivariate forecasting benchmarks at $h = 96$ (3-seed mean $\pm$ standard deviation). Bold marks the lowest mean in each column.

former has no public implementation, we reproduce it from the paper description. The results should therefore be interpreted as a controlled common-protocol comparison rather than an exhaustive per-model tuning study.

Training and implementation. All models are trained with Adam using a learning rate of 10^{-3} , MSE loss, and at most 15 epochs with early stopping. Frameformer uses $d_{\text{model}} = 128$, four Householder reflections per variable, two Transformer blocks, and a patch size of 16. Full implementation, optimization, and compute details are provided in Appendix A.

5.1 Main Results

As shown in Table 3, Frameformer achieves the lowest mean MSE on six of the eight benchmarks and ranks second on ETTh2, trailing the best mean by only 0.001. It ranks first on both PEMS8 and PEMS4, which are also the datasets where the controlled interface comparisons show the largest improvements over identity-based aggregation. On PEMS4, Frameformer reaches 0.111 MSE with approximately 0.79M parameters, compared with Crossformer’s 0.112 using approximately 8.7M parameters. Frameformer outperforms iTransformer, DLinear, and TimeXer on seven of the eight benchmarks, and outperforms PatchTST on seven with one tie.

5.2 Multi-Horizon Evaluation

We further evaluate whether Frameformer remains competitive beyond the default $h = 96$ setting. Figure 4 summarizes results at $h \in \{336, 720\}$ on ETTh1, ETTh2, Weather, and Solar. For each dataset and horizon, MSE is normalized by the worst-performing method, so 100% denotes the worst result and lower values are better.

Frameformer ranks first on ETTh2 and Solar at both horizons, and first or second on ETTh1. Its advantage is especially pronounced on Solar, where it remains substantially below all competing methods at both horizons. Weather is more tightly clustered: Frameformer ranks fourth at $h = 336$ and third at $h = 720$, with only small absolute differences among the leading methods. Complete numerical results for all evaluated dataset–horizon combinations are provided in Appendix F.

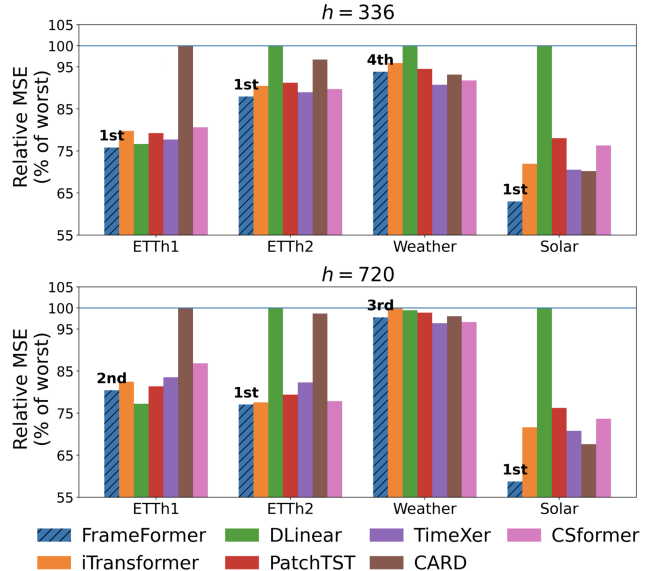


Figure 4: Forecasting performance at $h = 336$ and $h = 720$. MSE is normalized by the worst-performing method, so 100% denotes the worst result and lower is better.

5.3 Operating Regime

The benchmark results align with the controlled interface analyses in the preceding section. Frameformer is strongest on the coupled sensor datasets, where cross-variable routing is beneficial and variable-specific interfaces provide substantial additional gains. Its advantage, however, is not universal. Exchange provides the clearest boundary: DLinear obtains an MSE of 0.106, whereas Frameformer reaches 0.115. This result suggests that variable-specific interfaces are most beneficial when cross-variable mixing itself provides useful predictive information.

6 Additional Analysis

The predictive results establish the regimes in which Frameformer performs well. We now return to the operator in Eq. (1) and examine how its source and receiver interfaces contribute to the observed gains. We first isolate the effects of source-specific expression and receiver-specific interpretation, and then test whether independent pair-specific oper-

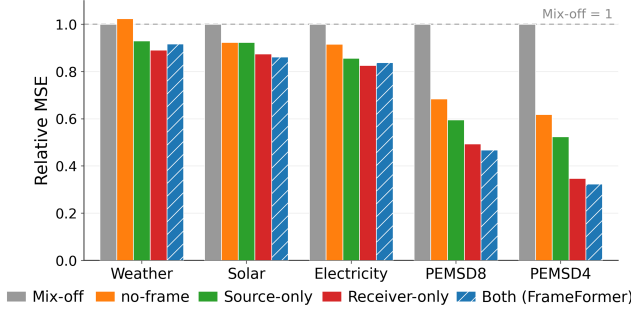


Figure 5: Value-path frame placement (Test MSE relative to Mix-off = 1).

ators provide useful additional capacity.

6.1 Where Does the Interface Gain Arise?

We next ask which component drives the interface gain: source-specific expression, receiver-specific interpretation, or their combination. Starting from shared-space mixing (no-frame) $\hat{\mathbf{v}}_i = \sum_j A_{ij} \mathbf{v}_j$, we compare a source-only interface $\hat{\mathbf{v}}_i = \sum_j A_{ij} \mathbf{U}_j \mathbf{v}_j$, a receiver-only interface $\hat{\mathbf{v}}_i = \mathbf{U}_i^\top \sum_j A_{ij} \mathbf{v}_j$, and the full tied interface Frameformer $\hat{\mathbf{v}}_i = \mathbf{U}_i^\top \sum_j A_{ij} \mathbf{U}_j \mathbf{v}_j$. The encoder, router, temporal stage, decoder, data splits, and optimization are otherwise fixed.

Figure 5 shows that receiver-specific interpretation accounts for most of the interface gain. The receiver-only interface reduces MSE by 5.2% on Solar, 27.6% on PEMS8, and 43.9% on PEMS4, recovering most of the improvement obtained by the full tied interface. It also slightly outperforms the full composition on Weather and Electricity. In contrast, the source-only interface is weaker than the receiver-only interface on five of the six datasets and provides no improvement on Solar.

Source-specific expression nevertheless provides a complementary gain in strongly coupled regimes. The full tied interface improves over the receiver-only interface on Solar, PEMS8, and PEMS4, by approximately 4.4% on average across these datasets, and achieves the lowest MSE on four of the six datasets overall. Thus, receiver-specific interpretation is the primary contributor, while source-specific expression provides additional value when cross-variable coupling is strong.

6.2 Do Variable-Specific Interfaces Suffice for Pairwise Value Transport?

Frameformer constructs each ordered-pair value transformation as $\mathbf{R}_{ij} = \mathbf{U}_i^\top \mathbf{U}_j$, using one k -reflection frame per variable and $O(Nkd)$ interface parameters. We test whether this endpoint factorization sacrifices useful expressivity using three relaxations, while holding the encoder, router, and decoder fixed: (i) *pair map* learns an independent dense operator for each ordered pair, (ii) *basis residual* adds pair-specific coefficients over a shared low-rank relation basis, and (iii) *edge residual* adds an independent low-rank residual per pair. The basis residual is evaluated at full N . Because

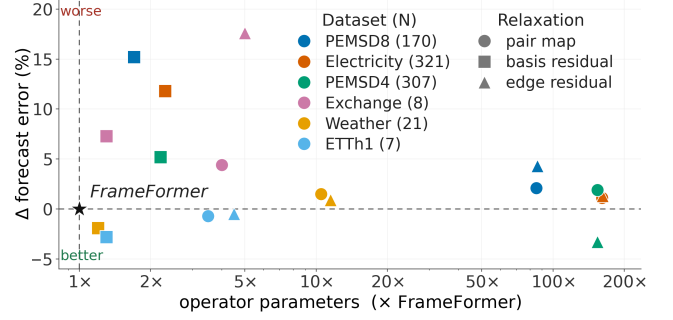


Figure 6: Accuracy–capacity trade-off of value-interface relaxations. Lower y is better. x shows interface parameters relative to FrameFormer on a log scale.

the other two alternatives scale quadratically with N , we use $N = 25$ for PEMS4, PEMS8, and Electricity and full N elsewhere, always comparing against the matched FrameFormer configuration.

As illustrated in Figure 6, none of the relaxations provides a consistent accuracy gain. At full N , the basis residual increases MSE by 5.2%–15.2% on PEMS4, Exchange, Electricity, and PEMS8, while improving Weather and ETTh1 by only 2%–3%. The more expensive pair-specific alternatives also mostly increase error. Their only notable gain is a 3.3% improvement from the edge residual on PEMS4 at reduced N , requiring $154\times$ as many interface parameters. Overall, the relaxations use up to $162\times$ as many parameters—53M versus 329k on Electricity—without a reliable benefit. Thus, $\mathbf{U}_i^\top \mathbf{U}_j$ is not merely a compressed approximation to independent pairwise operators, but a useful inductive bias.

7 Conclusion

We revisited cross-variable aggregation by distinguishing routing, which determines which sources contribute and how strongly, from value interfaces, which determine how their information is expressed and interpreted. FrameMix realizes this second design axis through variable-specific orthogonal frames, inducing source–receiver-specific value processing without learning an independent operator for every variable pair. Integrated into the patch-based Frameformer architecture, FrameMix improves cross-variable exchange while leaving the routing mechanism unchanged. Experiments across routing rules, learned and fixed frames, and alternative parameterizations show that the benefit arises broadly from variable-specific value processing rather than from additional model capacity. Receiver-side processing accounts for most of the improvement, while source-side expression provides further gains in strongly coupled regimes. More expressive pair-specific alternatives add substantial capacity without reliable accuracy benefits. Under a common training protocol, Frameformer performs strongly across eight benchmarks and forecast horizons up to 720, with its largest gains on coupled sensor datasets. Its weaker performance on Exchange further shows that value interfaces are not uniformly beneficial across forecasting regimes.

References

- Asif, M. S. H.; Khan, T.; and Khan, M. M. 2026. Temporal Sheaf Neural Networks with Dynamic Orthogonal Transport. *arXiv preprint arXiv:2606.10071*.
- Bamberger, J.; Barbero, F.; Dong, X.; and Bronstein, M. M. ??? Bundle Neural Network for message diffusion on graphs. In *The Thirteenth International Conference on Learning Representations*.
- Barbero, F.; Bodnar, C.; Sáez de Ocáriz Borde, H.; Bronstein, M.; Veličković, P.; and Liò, P. 2022a. Sheaf Neural Networks with Connection Laplacians. In *Topological, Algebraic, and Geometric Learning Workshops (PMLR 196)*.
- Barbero, F.; Bodnar, C.; Sáez de Ocáriz Borde, H.; and Liò, P. 2022b. Sheaf Attention Networks. In *NeurIPS 2022 Workshop on Symmetry and Geometry in Neural Representations*.
- Borgi, A.; Silvestri, F.; and Liò, P. 2025. Polynomial Neural Sheaf Diffusion: A Spectral Filtering Approach on Cellular Sheaves. *arXiv preprint arXiv:2512.00242*.
- Cao, D.; Wang, Y.; Duan, J.; Zhang, C.; Zhu, X.; Huang, C.; Tong, Y.; Xu, B.; Bai, J.; Tong, J.; and Zhang, Q. 2020. Spectral Temporal Graph Neural Network for Multivariate Time-Series Forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Kingma, D. P.; and Ba, J. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Liu, Y.; Hu, T.; Zhang, H.; Wu, H.; Wang, S.; Ma, L.; and Long, M. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *International Conference on Learning Representations (ICLR)*.
- Nie, Y.; Nguyen, N. H.; Sinthong, P.; and Kalagnanam, J. 2023. A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers. In *International Conference on Learning Representations (ICLR)*.
- Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Wang, H.; Mo, Y.; Xiang, K.; Yin, N.; Dai, H.; Li, B.; Fan, S.; and Mo, S. 2025. CSformer: Combining Channel Independence and Mixing for Robust Multivariate Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 39, 21090–21098.
- Wang, X.; Zhou, T.; Wen, Q.; Gao, J.; Ding, B.; and Jin, R. 2024a. CARD: Channel Aligned Robust Blend Transformer for Time Series Forecasting. In *International Conference on Learning Representations (ICLR)*.
- Wang, Y.; Wu, H.; Dong, J.; Qin, G.; Zhang, H.; Liu, Y.; Qiu, Y.; Wang, J.; and Long, M. 2024b. TimeXer: Empowering Transformers for Time Series Forecasting with Exogenous Variables. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wu, Z.; Pan, S.; Long, G.; Jiang, J.; Chang, X.; and Zhang, C. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*.
- Yi, K.; Zhang, Q.; Fan, W.; He, H.; Hu, L.; Wang, P.; An, N.; Cao, L.; and Niu, Z. 2023. FourierGNN: Rethinking Multivariate Time Series Forecasting from a Pure Graph Perspective. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zeng, A.; Chen, M.; Zhang, L.; and Xu, Q. 2023. Are Transformers Effective for Time Series Forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.
- Zhang, Y.; and Yan, J. 2023. Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting. In *International Conference on Learning Representations (ICLR)*.
- Zhao, L.; and Shen, Y. 2024. Rethinking Channel Dependence for Multivariate Time Series Forecasting: Learning from Leading Indicators. In *International Conference on Learning Representations (ICLR)*.

A Architecture and Experimental Details

A.1 FrameMix Architecture

Patch encoder and decoder. The length- L input for each variable is divided into $P = L/s$ non-overlapping patches of length s and projected into $\mathbb{R}^d$ by the shared patch embedding layer with positional embedding. After the blocks, each variable i has a representation $\mathbf{Z}_i \in \mathbb{R}^{P \times d}$; a single shared forecasting head flattens $\mathbf{Z}_i$ and maps $\mathbb{R}^{Pd} \rightarrow \mathbb{R}^H$, producing the H -step forecast. The horizon H is set independently of L : across the multi-horizon study, only the output dimension of the forecasting head changes, while the rest of the architecture remains unchanged. We use $L = 96$ throughout.

Two-stage transformer block. Each of the $n = 2$ blocks contains two sub-layers: (i) FrameMix across variables, applied independently at each patch position, followed by (ii) standard multi-head self-attention across patch positions within each variable. Each sub-layer uses a residual connection and layer normalization, followed by a shared feed-forward network.

Patch-position modulation. To allow each variable-specific frame to vary smoothly with patch position without introducing an explicit time warp, we parameterize the Householder vectors as

$$\mathbf{h}_{i,\ell}^{(p)} = \mathbf{h}_{i,\ell}^{\text{base}} + \sigma(\gamma_i) \text{pos}(p) \mathbf{h}_{i,\ell}^{\text{dyn}}, \quad \mathbf{U}_i^{(p)} = \prod_{\ell=1}^k \mathcal{H}(\mathbf{h}_{i,\ell}^{(p)}),$$

where $\text{pos}(p) \in [-1, 1]$ is a normalized scalar encoding of patch position p , and γ_i is a learnable per-variable gate initialized to -3 . Consequently, the dynamic component is largely suppressed at initialization and is learned during training.

A static variant, obtained by removing the modulation term, matches the patch-modulated variant within seed noise on PEMSD8, PEMSD4, Electricity, and ETTh1, while modulation improves Solar (.180 vs. .202). See Appendix G.3 for the full comparison between the static and patch-modulated variants.

Householder frame construction. Each variable-specific frame $\mathbf{U}_i \in \mathbb{R}^{d \times d}$ is parameterized as a product of k Householder reflections:

$$\mathbf{U}_i = \prod_{\ell=1}^k \mathcal{H}(\mathbf{h}_{i,\ell}), \quad \mathcal{H}(\mathbf{h}) = \mathbf{I} - \frac{2\mathbf{h}\mathbf{h}^\top}{\mathbf{h}^\top \mathbf{h}}.$$

Each reflection is orthogonal, so $\mathbf{U}_i$ is orthogonal for any $\mathbf{h}_{i,\ell}$. We use $k = 4$ reflections, requiring $O(kd)$ learnable parameters per variable.

Temporal carry and why it is disabled. FrameMix admits an auxiliary temporal carry path that reuses each variable’s frames to transport information across adjacent patch positions:

$$\hat{\mathbf{v}}_i^{(p)} = \mathbf{v}_i^{(p)} + \left(\mathbf{U}_i^{(p)}\right)^\top \mathbf{U}_i^{(p-1)} \mathbf{v}_i^{(p-1)}.$$

This path is *disabled* in all reported results. If enabled, each variable retains a stable frame association outside the cross-variable aggregation operator; consequently, shuffling the frame assignments within that operator would leave the carry

path unchanged, confounding the intended intervention. Disabling the carry ensures that the cross-variable FrameMix operator is the sole consumer of the frame parameters, so every reported result corresponds to a single, coherent architecture. Diagnostic tables are obtained from independent training runs and reproduce the main-table Frameformer within 0.005 absolute MSE (the largest difference occurs on Weather).

A.2 Training and Evaluation Protocol

Optimizer and schedule. All models are trained with Adam (Kingma and Ba 2014) ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$), learning rate 10^{-3} , no warm-up, for 15 epochs with early stopping on validation MSE (patience 5). MSE is the training and evaluation loss throughout.

Data splits and normalization. We follow the standard 7/1/2 train/val/test splits for ETT datasets, and the 6/2/2 splits used by iTransformer for Solar, PEMSD8/4, Weather, and Electricity. All inputs are normalized per-variable to zero mean and unit variance using training-set statistics.

Seed protocol. All results are reported as three-seed means; seeds 0, 1, 2 are used throughout. Paired statistical tests (Wilcoxon, paired- t) treat each dataset as one observation.

Model-specific hyperparameters. External baselines use paper-recommended configurations under common loaders. Frameformer uses latent dimension $d = 128$, $k = 4$ Householder reflections per frame, $n = 2$ blocks, patch length $s = 16$, and forecast horizon $H = 96$, unless otherwise stated.

Parameter counts and compute. Frameformer at PEMSD4 ($N = 307$, $d = 128$, $k = 4$): $\approx 0.79\text{M}$ parameters. Crossformer at PEMSD4: $\approx 8.7\text{M}$ parameters. All experiments were run on a single NVIDIA A100 40 GB GPU; training one Frameformer run at $H = 96$ takes approximately 8–12 minutes.

Minimal-operator interface protocol. The transplant, readaptation, Haar, component-hierarchy, and relation-operator experiments use a minimal faithful operator: a shared linear encoder, per-variable orthogonal Householder frames, a per-instance attention router, and a shared linear decoder. Its value-path output is verified against the corresponding full PyTorch (Paszke et al. 2019) operator to numerical tolerance (10^{-6}). Five independent seeds are used for directed transplantation. On PEMSD4 and PEMSD8, the variable count is capped at $N = 25$ for tractable all-pairs sweeps; ETTh1, Weather, and Exchange use their audited variable sets. This protocol isolates frame and component compatibility and is distinct from the full-forecaster predictive and placement evaluations.

A.3 Implementation Details

Attention and value-path implementation. At each patch position p , the cross-variable FrameMix operator replaces only the *value* aggregation path of a standard multi-head attention layer:

$$\hat{\mathbf{v}}_i^{(p)} = \left(\mathbf{U}_i^{(p)}\right)^\top \sum_{j=1}^N A_{ij}^{(p)} \mathbf{U}_j^{(p)} \mathbf{v}_j^{(p)},$$

where $A_{ij}^{(p)}$ denotes the post-softmax attention weight from source variable j to receiver variable i , computed in the standard manner from the query and key projections. Thus, FrameMix is a drop-in replacement for the value aggregation path, while the query-key computation and attention weights remain unchanged.

Shuffle controls. We use two shuffles that intervene on different components of the cross-variable mixer. To disrupt routing while preserving the distribution of routing weights within each receiver row, we sample a uniform permutation π over the variable indices and replace

$$A_{ij}^{(p)} \quad \text{with} \quad A_{i,\pi(j)}^{(p)}.$$

To disrupt variable-frame ownership while leaving the routing weights unchanged, we instead replace

$$\mathbf{U}_i^{(p)} \quad \text{with} \quad \mathbf{U}_{\pi(i)}^{(p)}.$$

During training, a new permutation is sampled at every forward pass. During evaluation, a new permutation is sampled for each batch, and the reported test-time shuffle result is averaged over five permutation draws per seed.

Identity-recovery numerical check. Setting all source and receiver interfaces to the identity, $\mathbf{S}_j = \mathbf{T}_i = \mathbf{I}$ for every i, j , in the iTransformer drop-in experiment exactly recovers the original iTransformer output, with zero numerical difference over 100 randomly generated inputs.

Reproducibility. All code and configurations will be released upon acceptance. Random seeds are fixed via `torch.manual_seed` and `numpy.random.seed` at the start of each run.

B Full Topology-Shuffle Analysis

B.1 Post-Hoc Adapter Results

A per-instance Pearson-correlation adapter is applied on top of four channel-independent backbones (DLinear, FITS, PatchTST, and CI-TimeMixer) and one cross-variable backbone (iTransformer), across five datasets (ETTh1, ETTh2, Weather, Solar, and Electricity) and four forecast horizons ($H \in \{96, 192, 336, 720\}$), giving 80 model $\times$ dataset $\times$ horizon cells. For each cell, we compare three adapter variants: real (learned structure), shuffled (column-permuted per instance), and identity (adapter removed). TOST equivalence at a 5% margin holds in 87.5% of the cells, and no variant dominates systematically. PIR improves its PatchTST backbone by 2%–9% MSE; the gain is fully retained under a per-instance shuffle of its cross-channel attention and is slightly enhanced on ETTh2.

B.2 Synthetic Sensitivity Validation

To verify that the shuffle control detects genuine structure when present, we construct two synthetic data-generating processes (DGPs) with installed cross-variable dependencies and confirm that the control produces the expected signature.

Clustered DGP. Variables are partitioned into clusters of size 4. Within each cluster, all series share a latent factor plus independent noise. An iTransformer trained on this data exploits the cluster structure, incurring a test-time shuffle

Dataset	learned	frozen rand.	identity	U-shuf
ETTh1	.455 $\pm$.004	.457 $\pm$.003	.462 $\pm$.003	.515 $\pm$.018
Weather	.149 $\pm$.002	.150 $\pm$.002	.169 $\pm$.003	.226 $\pm$.011
Solar	.180 $\pm$.005	.202 $\pm$.009	.192 $\pm$.006	.228 $\pm$.014
PEMSD8	.178 $\pm$.002	.211 $\pm$.005	.257 $\pm$.007	.517 $\pm$.032
PEMSD4	.111 $\pm$.001	.153 $\pm$.004	.212 $\pm$.006	.650 $\pm$.041
Electricity	.140 $\pm$.003	.143 $\pm$.003	.153 $\pm$.004	.300 $\pm$.019

Table 4: Frame presence: test MSE ($h = 96$, 3-seed mean $\pm$ std).

penalty of +331.7%. Under train-time shuffle, the penalty remains substantial (+287.3%), confirming that the learned routing structure is both committed to and necessary in this DGP, in contrast to the real-data setting where train-time shuffle recovers normal accuracy.

Directed-coupling DGP. Variable j causally influences variable i with a one-step lag, and only these directed dependencies are informative. The iTransformer exploits this structure (test-time shuffle +29.4%); train-time shuffle degrades performance similarly (+24.1%), again confirming that the installed structure is necessary. Across both DGPs, the control clearly separates the genuine-structure regime from the substitutable regime observed on real datasets, validating its sensitivity.

B.3 Within-Host Topology Checks

To confirm that the topology conclusion also holds within the FrameMix architecture, we perform routing-weight shuffle controls by permuting the routing matrix $\mathbf{A}^{(p)}$ while evaluating the shared-frame ($\mathbf{U}_i^{(p)} = \mathbf{I}$) and variable-specific-frame (FrameMix) variants separately.

In both cases, train-time routing-weight shuffling recovers within 2% of normal performance on all six datasets, whereas test-time routing-weight shuffling substantially degrades performance (+5%–65%, depending on the dataset). This rules out the possibility that the topology-substitutability finding is an artifact of the iTransformer or Crossformer architecture.

C Full Frame Controls

C.1 Frame Presence

The full three-seed results extend the frame-presence comparison in Table 4 by reporting standard deviations and the test-time frame-assignment shuffle condition. The learned-identity and learned-frozen-random gaps are consistent in direction across all three seeds on every dataset. The frozen-random-identity comparison is directionally consistent on five of the six datasets; Solar reverses on one seed, consistent with the observation in the main paper.

C.2 Frame Assignment

T1/T2 details. For T1 (test-time frame-assignment shuffle), we sample a fresh permutation π over the N variables for each evaluation batch and replace

$$\mathbf{U}_i^{(p)} \leftarrow \mathbf{U}_{\pi(i)}^{(p)} \quad \text{for all } i \in \{1, \dots, N\} \text{ and } p \in \{1, \dots, P\}.$$

Donor frames	Model 0	Model 1	Model 2	Model 3	Model 4
$\mathcal{U}^{(0)}$	.18	1.07	.87	.82	.92
$\mathcal{U}^{(1)}$	.96	.18	.88	.84	1.04
$\mathcal{U}^{(2)}$	.98	.91	.19	.93	.89
$\mathcal{U}^{(3)}$	1.07	.81	1.09	.18	.77
$\mathcal{U}^{(4)}$	.87	1.20	.85	1.01	.19

Table 5: Directed frame transplantation on PEMSD4. Entry (a, b) is the test MSE of recipient model b after replacing its frame set with donor model a ’s frame set $\mathcal{U}^{(a)}$. The diagonal mean is .184, while the off-diagonal mean is .939 (5.11 $\times$ higher).

Dataset	attention Q/K	frames	decoder head
PEMSD4	1.13 $\times$	5.11 $\times$	6.91 $\times$
Exchange	—	0.78 $\times$	3.50 $\times$

Table 6: Component-transplant hierarchy: mean off-diagonal/diagonal MSE ratio when one component is transplanted across five seeds (1.0 $\times$ = no penalty; Q/K not run on Exchange).

The reported result is averaged over five independent permutation draws per seed.

For T2 (train-time frame-assignment shuffle), a fresh permutation π is sampled at every training step and applied in the same manner. The model is trained to convergence and then evaluated using its own learned variable-specific frames, without frame reassignment at test time.

Gain-lost calculation. Let $\mathcal{E}_{\text{normal}}$, $\mathcal{E}_{\text{T2}}$, and $\mathcal{E}_{\text{identity}}$ denote the test MSEs of the normally trained Frameformer, the T2 model, and the identity-frame baseline, respectively. We measure the fraction of the learned-frame improvement removed by train-time reassignment as

$$\text{GainLost} = \frac{\mathcal{E}_{\text{T2}} - \mathcal{E}_{\text{normal}}}{\mathcal{E}_{\text{identity}} - \mathcal{E}_{\text{normal}}}.$$

A value of 1 indicates that T2 removes the entire improvement over the identity-frame baseline.

Reshuffle consistency check. For each T2 model, we apply a fresh test-time frame-assignment shuffle, denoted T2-reshuf, and verify that its test MSE differs from $\mathcal{E}_{\text{T2}}$ by at most 0.001. This agreement confirms that the T2 model has lost its dependence on a specific variable-frame assignment rather than memorizing the distribution of training-time permutations.

D Co-Adapted Interface Experiments

All experiments in this section use the minimal faithful operator described in Appendix A.2. The full Frameformer establishes predictive performance and the importance of persistent variable-frame assignment, whereas the experiments below isolate the compatibility between variable-specific frames and neighboring model components across independently trained runs.

Dataset (N)	Variant	Acc. N	Param ratio	Δerr (%)
PEMSD8 (170)	pair map	25	85 $\times$	+2.1
	basis residual	170	1.7 $\times$	+15.2
	edge residual	25	86 $\times$	+4.3
PEMSD4 (307)	pair map	25	154 $\times$	+1.9
	basis residual	307	2.2 $\times$	+5.2
	edge residual	25	154 $\times$	−3.3
Electricity (321)	pair map	25	161 $\times$	+1.1
	basis residual	321	2.3 $\times$	+11.8
	edge residual	25	162 $\times$	+1.3
Weather (21)	pair map	21	10.5 $\times$	+1.5
	basis residual	21	1.2 $\times$	−1.9
	edge residual	21	11.5 $\times$	+0.9
Exchange (8)	pair map	8	4.0 $\times$	+4.4
	basis residual	8	1.3 $\times$	+7.3
	edge residual	8	5.0 $\times$	+17.6
ETTh1 (7)	pair map	7	3.5 $\times$	−0.7
	basis residual	7	1.3 $\times$	−2.8
	edge residual	7	4.5 $\times$	−0.5

Table 7: Relation-operator relaxations behind Figure 6. Δerr = test-MSE change vs. Frameformer (lower is better); the quadratic variants use a capped $N=25$ for accuracy but full- N parameter ratios (so “53M vs. 329k” is a full- N figure).

D.1 Five-Seed Directed Transplantation on PEMSD4

Table 5 reports the directed transplantation matrix at full precision. Let

$$\mathcal{U}^{(a)} = \left\{ \mathbf{U}_i^{(p,a)} \right\}_{i=1,p=1}^{N,P}$$

denote the complete set of patch-dependent variable frames learned by donor model a . Entry (a, b) reports the test MSE of recipient model b after replacing its frame set $\mathcal{U}^{(b)}$ with donor frame set $\mathcal{U}^{(a)}$, while keeping all other parameters of recipient model b fixed. All models use the same dataset, architecture, and training protocol but are trained with independent random seeds.

All 20 directed off-diagonal transplantations increase test error. The matrix is not symmetric because the non-frame parameters of the recipient models differ across independently trained runs.

D.2 Foreign-Frame Readaptation

Two normally trained PEMSD4 models obtain test MSEs of .1798 (model A) and .1802 (model B). Let $\mathcal{U}^{(A)}$ and $\mathcal{U}^{(B)}$ denote the complete sets of variable-specific frames learned by models A and B, respectively. Replacing only model B’s frame set $\mathcal{U}^{(B)}$ with $\mathcal{U}^{(A)}$, while keeping all other parameters of model B fixed, increases the test MSE to .9763. Randomly permuting the variable ownership of model B’s own frames produces an MSE of .7962.

We then fix the transplanted donor frame set $\mathcal{U}^{(A)}$ and retrain all remaining parameters of model B, obtaining an MSE of .1789. Thus, the same donor frames that fail under

Dataset	sync residual	FM–data residual
Exchange	.337	.637
Electricity	.431	.723
ETTh1	.481	.827
Weather	.612	.968
PEMSD4	.737	.951

Table 8: Static transport audit (lower = closer FrameMix–data match or greater synchronizability).

Dataset	shared	FrameMix	difference	norm ratio
ETTh1	.53	.43	-.10	.76
Weather	.55	.39	-.16	.85
Electricity	.67	.38	-.29	.60
Exchange	.92	.48	-.45	.96
PEMSD4	.87	.25	-.63	.41

Table 9: Cosine compatibility of the aggregate with the target representation; FrameMix lowers it.

post-hoc transplantation support a fully accurate model once the surrounding modules are allowed to readapt.

D.3 Identity, Haar, and Learned Refit Decomposition

On PEMS4, the frame-free reference obtains an MSE of .921. Refitting the surrounding model under the fixed identity convention

$$\mathbf{U}_i^{(p)} = \mathbf{I} \quad \text{for all } i \in \{1, \dots, N\} \text{ and } p \in \{1, \dots, P\}$$

reduces the MSE to .395, recovering 71% of the absolute gap between the frame-free reference and learned FrameMix. Using fixed Haar-random frame sets, four independent draws obtain $.277 \pm .002$, corresponding to an 87% recovery, while learned frames reach .184. This comparison separates the contributions of readaptation, a persistent orthogonal scaffold, and learned frame selection.

On PEMS8, learned frames do not improve over fixed Haar-random frames beyond seed variation: the learned-frame model obtains $.300 \pm .027$, the Haar-fixed model obtains $.296 \pm .010$, and a learned model trained for 20 epochs obtains .301. Thus, the benefit of learned frame selection is dataset-dependent, whereas the usefulness of a persistent variable-specific frame assignment replicates across these settings.

D.4 Component-Transplant Hierarchy

Repeating the five-seed transplantation analysis for individual model components gives the mean ratio between off-diagonal and matched test MSE summarized in Table 6.

The PEMS4 result shows that cross-seed transplantation failure is not a generic property of all learned parameters. The Exchange result further shows that decoder non-transferability can persist even in a regime where compatibility between variable-specific frames is not predictive of forecasting performance.

Dataset	no-mix $\mathbf{v}$	shared	FrameMix
PEMSD4	6.30	4.19	30.77
Exchange	4.81	4.85	17.39
ETTh1	26.22	26.58	42.82
Weather	9.82	12.74	36.38

Table 10: Effective rank of latent values: no-mix input, after shared-map mixing, and after FrameMix.

D.5 Relation-Operator Audit

We compare FrameMix with three increasingly expressive alternatives. The relation-only model replaces the node factorization by a learned ordered-pair operator. The residual-basis model inserts

$$\mathbf{E}_{ij} = \mathbf{I} + \sum_b c_{ijb} \mathbf{B}_b$$

between the source and receiver maps, and the full-edge model gives every ordered pair an independent low-rank residual. The residual coefficients are initialized at zero. Unit tests verify that this setting reproduces FrameMix exactly (maximum forward difference 0.00) and that all relation parameters receive gradients.

The results are reported in Table 7. The learned residuals are not dormant: on PEMS4 and Weather, mean $\|\mathbf{E}_{ij} - \mathbf{I}\|_F$ lies between 2.7 and 4.3 and the fraction of near-identity pairs is zero. Nevertheless, both the residual-basis and full-edge models substantially degrade PEMS4. This establishes that the failed extension is not an implementation artifact or an unused branch.

E Static-Geometry Audit

The following analyses test, rather than assume, that learned frames recover an intrinsic latent geometry. They are retained as a negative mechanism audit because they delimit the interpretation of the learned maps. The PEMS setting in this audit is PEMS4.

E.1 Pairwise Transport and Synchronization

For each variable pair, we estimate an independent orthogonal Procrustes map from latent representations and compare it with the learned relative map $\mathbf{R}_{ij} = \mathbf{U}_i^\top \mathbf{U}_j$. We also measure the residual of their best node-factorized synchronization.

Table 8 summarizes both quantities. Across all datasets, the synchronization residual remains substantial, and the learned relative maps also exhibit large FrameMix–data residuals. Thus, the learned interfaces provide little evidence for a shared latent geometry that is consistent across variables.

Table 9 further compares the cosine compatibility between the aggregated representation and the target representation under shared-space mixing and FrameMix. Surprisingly, FrameMix consistently *reduces* this compatibility across all datasets. Thus, the gains from FrameMix cannot be explained by making the aggregated representation more

Dataset	Frameformer	iTransf.	DLinear	PatchTST	TimeXer	CARD	CSformer
<i>Horizon $H = 192$</i>							
ETTh1	.511	.524	.516	.539	.530	.655	.512
ETTh2	.318	.300	.324	.303	.310	.401	.305
Exchange	.233	.228	.206	.247	.236	.344	.247
Weather	.206	.226	.239	.224	.209	.211	.219
Solar	.203	.237	.312	.242	.230	.225	.227
Electric	.158	.170	.195	.179	.159	—	—
<i>Horizon $H = 336$</i>							
ETTh1	.555	.584	.561	.580	.569	.732	.590
ETTh2	.350	.360	.398	.363	.354	.385	.357
Exchange	.504	.408	.438	.455	.419	.608	.417
Weather	.273	.279	.291	.275	.264	.271	.267
Solar	.218	.249	.346	.270	.244	.243	.264
Electric	.181	.184	.208	.195	.177	—	—
<i>Horizon $H = 720$</i>							
ETTh1	.678	.695	.651	.686	.704	.843	.732
ETTh2	.456	.459	.592	.470	.487	.584	.461
Exchange	1.489	1.058	1.538	1.172	1.149	.968	1.103
Weather	.349	.357	.355	.353	.344	.350	.345
Solar	.205	.250	.349	.266	.247	.236	.257
Electric	.221	.222	.243	.234	.216	—	—

Table 11: Multi-horizon test MSE (3-seed mean), $H \in \{192, 336, 720\}$. Bold indicates the lowest MSE in each row. Crossformer is omitted because it diverged on ETTh2 and Exchange. CARD and CSformer were not evaluated on Electricity (—).

closely aligned with the receiver representation in the original latent space.

Attention weighting does not rescue the account: uniform/attention-weighted residuals are .827/.756 (ETTh1), .723/.706 (Electricity), .968/.927 (Weather), .637/.637 (Exchange), and .951/.952 (PEMSD4). Neither PEMS4 nor the Exchange control shows preferential fit on high-attention edges.

E.2 Effective Rank and the No-Mix Baseline

Table 10 compares the effective rank of the latent values before cross-variable aggregation (no-mix), after shared-space aggregation, and after FrameMix. Across all datasets, shared-space aggregation changes the effective rank only marginally, whereas FrameMix substantially increases it. This confirms that assigning variable-specific orthogonal frames diversifies the latent value representations. However, the subsequent shuffle experiment shows that this rank expansion alone cannot explain the predictive gains, since test-time frame reassignment preserves or even increases the effective rank while substantially degrading forecasting accuracy.

F Multi-Horizon Evaluation

F.1 Full Multi-Horizon Benchmark Results

Table 11 reports the full $H \in \{192, 336, 720\}$ benchmark. Frameformer stays competitive across horizons and is dominant on Solar; TimeXer becomes competitive on Weather and Electricity at long range, and Frameformer stays weak on Exchange.

The main exception is Exchange at $H = 720$, where Frameformer (1.489) falls behind iTransformer (1.058) and

CARD (.968). This is the regime where cross-variable coupling is weakest; the variable-specific interface does not help here and appears to fit long-range noise, though we do not isolate the precise cause.

F.2 PEMS Horizon Curves

We report Frameformer at all four horizons on the PEMS datasets; the baselines were not rerun at long horizons under our compute budget. Test MSE grows gradually with horizon—PEMSD8 .178/.257/.316/.359 and PEMS4 .111/.126/.133/.158 at $H = 96/192/336/720$ —without the sharp long-horizon degradation seen on Exchange. The interface thus remains useful precisely where cross-variable coupling is strong.

G Additional Parameterization Controls

G.1 Affine Interface Control

To verify that the frame gain does not merely reflect the Householder parameterization, we replace the orthogonal source and receiver frames with independent per-variable affine maps $\mathbf{S}_j(\mathbf{v}) = \mathbf{s}_j^{\text{src}} \odot \mathbf{v} + \mathbf{b}_j^{\text{src}}$ and $\mathbf{T}_i(\mathbf{z}) = \mathbf{s}_i^{\text{tgt}} \odot \mathbf{z} + \mathbf{b}_i^{\text{tgt}}$ inside FrameMix, giving

$$\hat{\mathbf{v}}_i = \mathbf{T}_i \left(\sum_{j=1}^N A_{ij} \mathbf{S}_j(\mathbf{v}_j) \right).$$

The two maps use $4d$ parameters per variable. Since the deployed Frameformer uses patch-modulated frames ($\approx 2kd$ frame parameters per variable at $k = 4$), the affine map is a *lower-parameter*, non-orthogonal control rather than an exactly parameter-matched one; it remains far below one

Dataset	vanilla	FM-learned	FM-affine	affine vs. $\mathbf{v}$
PEMSD8	.361 $\pm$.001	.349 $\pm$.004	.309 $\pm$.012	−14.3%
PEMSD4	.309 $\pm$.002	.186 $\pm$.004	.164 $\pm$.005	−47.1%
Weather	.175 $\pm$.001	.163 $\pm$.002	.162 $\pm$.001	−7.5%
ETTh1	.470 $\pm$.001	.475 $\pm$.008	.470 $\pm$.001	−0.1%

Table 12: Drop-in transfer to iTransformer ($h = 96$, 3-seed mean $\pm$ std). Both per-variable maps improve vanilla on PEMS and Weather but not ETTh1; the orthogonal margin inverts relative to Frameformer.

dense operator per ordered pair. That a lightweight affine map matches the orthogonal frame strengthens the conclusion that the load-bearing factor is a persistent per-variable transformation rather than the orthogonal form specifically.

Results (Table 2): the affine control recovers 91%–94% of the identity gap on the PEMS datasets and 83% on Weather, but only 38% on Solar. It slightly outperforms the orthogonal frame on Electricity and is outperformed on the other four datasets. The orthogonal–affine margin is small overall and not statistically significant (paired Wilcoxon over the five datasets in Table 2, two-sided $p = 0.125$); the ordering further reverses in the iTransformer drop-in below. The true capacity-matched control appears in the iTransformer drop-in experiment (Appendix G.2), where a parameter-matched vanilla scale-up improves PEMS4 by only 3.6% vs. 47.1% for the affine correction. Thus the robust conclusion is not that orthogonality is universally optimal, but that per-variable transformation around the mixer is the load-bearing intervention.

G.2 Drop-In Transfer to iTransformer

To confirm the correction is not load-bearing only inside an architecture designed around it, we insert it directly into iTransformer’s variate attention, replacing its value path with

$$\hat{\mathbf{v}}_i = \mathbf{T}_i \left(\sum_{j=1}^N A_{ij} \mathbf{S}_j(\mathbf{v}_j) \right)$$

at full d -dimensional width, using the same independent source and receiver maps $\mathbf{S}_j, \mathbf{T}_i$ as above; setting $\mathbf{S}_j = \mathbf{T}_i = \mathbf{I}$ recovers the original iTransformer numerically (0 output difference, verified). The resulting performance is summarized in Table 12.

The value-interface intervention transfers most strongly where the frame-ablation gap was largest: the affine variant improves iTransformer by 47.1% on PEMS4 and 14.3% on PEMS8; ETTh1 remains within 1%, the weak-frame-gain regime. The affine variant outperforms the orthogonal one inside iTransformer, confirming that the transferable claim concerns variable-specific maps around the mixer rather than the Householder form.

G.3 Static vs. Patch-Modulated Frames

Table 13 summarizes the comparison between static and patch-modulated FrameMix. Solar is the only dataset with a material gap between the static and patch-modulated learned

Dataset	Static	Modulated
ETTh1	.455	.455
Weather	.150	.149
Solar	.202	.180
PEMSD8	.179	.178
PEMSD4	.112	.111
Electricity	.141	.140

Table 13: Static vs. patch-modulated FrameMix (test MSE, $h = 96$, 3-seed mean).

frames. Because a fixed Hadamard frame also performs well on Solar (Table 1), this comparison does not isolate patch dependence as the underlying mechanism; we therefore report it as an implementation-specific exception rather than assigning it a causal interpretation. On all other datasets the static and modulated variants agree within seed noise. The main paper uses the modulated variant throughout.